CS 450: Computer Networks
Lab 9_2: Semester Project Kickoff

Instructions: As a team, complete the tasks below and share the results with all team members. Each student must submit their finalized document on Blackboard. Take photos or screenshots for each completed "Part" as instructed and paste them into the designated box below that part. **The purpose of this lab is to begin documentation of your team project.**

Hardware and Software Role Mapping

- **HMI Pi (Raspberry Pi 5 + Touchscreen Display):** Primary control station, live user interface (GUI, dashboard, or structured terminal UI) host, and local testing console.
- **Sense HAT Pi (Standalone Pink/White Raspberry Pi 5):** Remote edge node equipped with Sense HAT environmental sensors, IMU, LED matrix, and joystick input.
- **Classroom Infrastructure and Edge Devices:** Managed Ubiquiti table switch, classroom LAN (eth0 / wlan0), remote storage devices (NAS database, MQTT broker, or SFTP/SCP backup server), and optional cross-table Raspberry Pis or PLCs.
- **Version Control and Documentation Platform:** Shared team GitHub repository and project documentation template.

Activity Overview

Over the final weeks of the semester, your team will design, implement, document, and demonstrate a distributed **Cyber-Physical System (CPS)** or networked edge application utilizing the classroom network and edge devices. While the Project Description establishes mandatory baseline constraints, such as utilizing at least six distinct TCP/UDP sockets, distributing software across multiple nodes, integrating real-world physical or system-level inputs, displaying a live HMI interface, logging persistent remote data, and deploying at least one Docker container, your team gets to decide the specific theme, functionalities, and architecture of your system.

In software and network systems engineering, successful implementation begins with translating high-level *baseline constraints* into concrete, measurable, and testable *project-specific requirements*. In this lab, your team will complete the foundational engineering specifications corresponding to **Sections 1, 2, and 3** of your final project documentation:

1. **Part 1 (Project Overview):** Define your project title, system purpose, operational scope, and hardware node role assignments, and initialize your shared team GitHub repository.
2. **Part 2 (Project Functional Requirements):** Translate each of the six mandatory generalized functional requirements into specific, measurable functional requirements tailored to your project.
3. **Part 3 (Project Non-Functional Requirements):** Define specific operational, port-binding, network hygiene, CLI diagnostic, and 3-minute turnkey reproducibility requirements for your architecture.

Pre-Lab Setup

1. Sit with your project partner at your assigned sandbox table and power on your **HMI Pi** and **Sense HAT Pi**.
2. Verify both nodes are reachable on the classroom LAN (eth0 / wlan0) and record your baseline table hardware configuration:

- **Team Member Names:** ___Isaac Gentry, Cameron Willoughby_____
- **Assigned Sandbox Table Number:** __1_____
- **HMI Pi IP Address (eth0 / wlan0):** _____etho_____
- **Sense HAT Pi IP Address (eth0 / wlan0):** _____eth0_____
- **Additional Devices Used (Other Table Pis / PLCs, if any):** _____

Part 1: Project Title, Purpose, and Scope (Project Description, Section 1)
In **Section 1 (Project Overview)** of the *Project Description*, your team is tasked with designing a distributed Cyber-Physical System (CPS) or networked edge application (for example: an automated industrial data pipeline and alerting platform, a distributed environmental control system, a network security/intrusion anomaly monitor, or a multi-node edge computing pipeline) culminating in a live demonstration during the **Week 15 Project Expo**.

Before writing code or opening sockets, your team must clearly define *what* your system does (**Purpose**) and *what boundaries and hardware components* it encompasses (**Scope**).

1. Define Your Project Title

Official Project Title: _____Network traffic monitor and controller_____

Application Category (Ex: Industrial Telemetry, Environmental Control, Security Monitor, Custom CPS): _____Security Monitor_____

2. Draft Section 1.1: Project Purpose (Problem Statement and Objectives)
Write a summarizing paragraph (4–6 sentences) defining the real-world or cyber-physical problem your project solves, who the intended user/operator is, and what high-level value the system provides:

Project Purpose Statement:
Industrial control environments in small manufacturing plants, quarries, and utilities often depend on edge devices that use unauthenticated network protocols and can't run endpoint security software, so any host that reaches the network can probe those devices or send commands that change physical outputs. These organizations rarely have dedicated security staff, which means a rogue or compromised host can disrupt operations long before anyone notices. This project builds a distributed edge security system in which a Raspberry Pi edge node streams live environmental sensor data and accepts actuator commands, while a central controller on the HMI Pi monitors for malicious behavior such as canary port probes, commands from unauthorized hosts, and

repeated rejected access attempts. When a threat is detected, the controller automatically orders an enforcement agent on the edge node to firewall the offending host within seconds, and the operator can review alerts and release quarantined hosts from a live touchscreen interface. The intended operator is a small IT or OT team, and the system's value is fast, automated containment through centralized policy and distributed enforcement, with every security event logged to remote storage for later review.

3. Draft Section 1.2: Project Scope & Hardware Role Mapping
Define the operational boundaries of your system by specifying exactly what each hardware node in your topology will do (and any external classroom resources you will integrate):

- **HMI Pi Role (Services hosted, UI role, data processing):** _____Quarantine controller, telemetry receiver, command client, logger, Displays logs, evaluates incoming data if quarantine needs to be sent_____
- **Sense HAT Pi Role (Sensors/actuators polled, daemons hosted):** _____Field Device being attacked_____
- **Remote Storage / Infrastructure Role (NAS Database, MQTT Broker, or SFTP/SCP Server):** _____Logger_____
- **Optional Additional Nodes (Other Sandbox Table Pis or PLCs—note table consultation if applicable):** _____
- **Out-of-Scope Boundaries (List 1–2 features your team is intentionally excluding to ensure completion by Week 15):** _____Layer 2 attacks and mitigations, real PLC's, Network Level enforcement_____

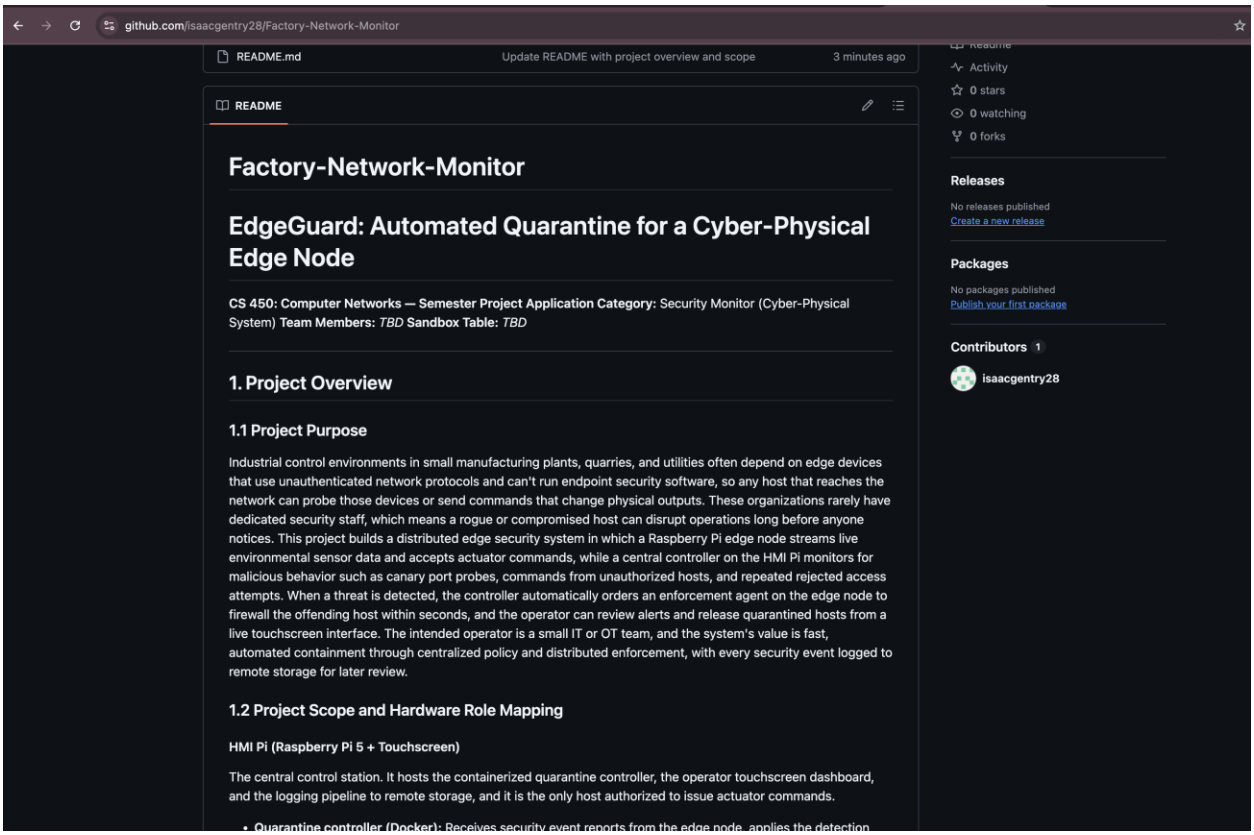
4. Initialize Your Shared Team GitHub Repository
Create your team's shared GitHub repository (with both partners added as collaborators) and commit an initial README.md containing your **Project Title**, **Purpose**, and **Scope** from Steps 1–3 above.

Shared GitHub Repository URL: _____ https://github.com/isaacgentry28/Factory-Network-Monitor.git _____

Question 1: Why is it critical in distributed network systems engineering to establish a clear boundary between *in-scope* and out-of-scope features before allocating sockets and writing application code?
Answer:

Because of two main reasons, scope creep and vulnerabilities. Defining clearly every single socket that will be open and used leaves no room for error or sockets being opened and not closed. Having open sockets leaves an edge node open and vulnerable for attack. Scope creep is a more project management tuned reason as well. Clearly defining scope helps keep the team on track, focused, and meeting certain deadlines that have been communicated clearly to key stakeholders.

PASTE SCREENSHOT OF PART 1 BELOW:
(Screenshot of your initialized GitHub repository showing your README.md with Section 1: Project Title, Purpose, and Scope)



Part 2: Specific Functional Requirements Specification (Section 2)
A **Functional Requirement** defines a specific behavior, data flow, input/output transformation, or function that the system must perform. In Section 2 of the Project Description, six generalized baseline functional requirements are mandated.

In this section, your team must translate each generalized requirement into a **detailed, project-specific functional requirement** using concrete engineering parameters (exact transport protocols, sensor types, sampling rates, UI controls, storage mechanisms, and container roles).

1. Six (6) Distinct TCP/UDP Network Sockets (No WebSockets)1. Six (6) Distinct TCP/UDP Network Sockets (No WebSockets)

- *Mandatory Baseline:* The project must use at least six (6) distinct TCP and/or UDP network sockets across the system architecture. (No WebSockets)
- **Write your Project's Specific Functional Requirement(s)** (List each of your 6+ planned sockets, specifying whether it uses raw TCP or UDP, which node acts as Server vs. Client, and what exact application payload it carries, and confirming zero WebSockets are used):

Socket 1 (TCP/UDP): ____ (UDP): Telemetry stream. Server: HMI Pi on port 5005. Client: Sense HAT Pi sensor daemon. Payload: one JSON datagram, containing node ID, timestamp, temperature, humidity, pressure, and sequence number

_____

Socket 2 (TCP/UDP): _____ (TCP): Actuator command channel. Server: Sense HAT Pi command service on port 5010. Client: HMI Pi command client (the only authorized

source). Payload: newline-delimited JSON commands (command type, LED pattern or setpoint value, request ID), each answered with a JSON acknowledgment (request ID, status: executed or rejected). _____
Socket 3 (TCP/UDP): ____ (TCP): Security event reporting. Server: HMI Pi quarantine controller on port 5020. Client: Sense HAT Pi event reporter. Payload: newline-delimited JSON events (event type such as canary_hit or unauthorized_command, source IP, source port, timestamp, details). _____
Socket 4 (TCP/UDP): _____ (TCP): Quarantine enforcement channel. Server: Sense HAT Pi enforcement agent on port 5030. Client: HMI Pi quarantine controller. Payload: newline-delimited JSON orders (action: quarantine or release, target IP, reason, order ID), each answered with a JSON confirmation of whether the firewall rule was applied.
_____
Socket 5 (TCP/UDP): _____ (UDP): Heartbeat and joystick events. Server: HMI Pi on port 5040. Client: Sense HAT Pi. Payload: JSON heartbeat datagrams every 2 seconds (node ID, timestamp, uptime, active quarantine count), plus a JSON datagram on each joystick press (event type, direction, timestamp). _____
Socket 6 (TCP/UDP): _____ (TCP): Canary listener. Server: Sense HAT Pi on port 5060. Client: none legitimate; any connecting host is treated as a scanner. Payload: none expected. The service records the source IP and port, closes the connection, and reports a canary_hit event over Socket 3.
_____
Socket 7 : (TCP): Remote logging. Server: classroom NAS database (port 3306 for MySQL or 5432 for PostgreSQL, depending on the NAS). Client: HMI Pi logger. Payload: SQL inserts of telemetry records and security event records.
List your specific functional requirements here:

2. Multi-Node Distributed Software Execution

- *Mandatory Baseline:* Software components must be distributed across both the HMI Pi and the Sense HAT Pi. (You can also use Pis at other sandbox tables for your project. If the other Pis are being used, consult with the team at that table before use. Other devices, such as PLCs can also be used if your team has experience with them.)
- **Write your Project's Specific Functional Requirement(s)** (Name the exact scripts, daemons, or services that will execute on the HMI Pi versus the Sense HAT Pi, and any additional table Pis/PLCs, and describe how they interact across the network):

HMI Pi Software Components: _____ the HMI Pi runs the containerized quarantine controller, a telemetry and heartbeat receiver, an authorized command client, a NAS logger, and the touchscreen dashboard. _____
Sense HAT Pi Software Components: _____ the Sense HAT Pi runs a sensor telemetry daemon, a source-restricted actuator command service, a canary listener, a root-privileged nftables enforcement agent, an event reporter, and a heartbeat and joystick sender, with the LED matrix showing system security state.
_____
Other Nodes / PLC Components (if applicable): _____
List your specific functional requirements here:List your specific functional requirements here:

- **FR-2.1:** System software shall be distributed across two nodes: the HMI Pi shall run the quarantine controller, telemetry receiver, command client, NAS logger, and touchscreen dashboard, and the Sense HAT Pi shall run the sensor daemon, command service, canary listener, enforcement agent, event reporter, heartbeat sender, and LED status driver.
- **FR-2.2:** The quarantine controller on the HMI Pi shall be the sole decision point for quarantine and release. The enforcement agent on the Sense HAT Pi shall apply firewall rules only in response to orders received over Socket 4 and shall never quarantine a host on its own.
- **FR-2.3:** The Sense HAT Pi shall detect security events locally (unauthorized commands on Socket 2 and connections to the canary listener on Socket 6) and forward them to the HMI Pi over Socket 3, so that detection happens at the edge and decisions happen centrally.
- **FR-2.4:** The enforcement agent shall maintain an allowlist containing the HMI Pi and the team's management host, and shall refuse and log any quarantine order targeting an allowlisted IP.
- **FR-2.5:** The HMI Pi shall be the only host whose actuator commands are executed by the Sense HAT Pi command service.
- **FR-2.6:** While any host is quarantined, the Sense HAT Pi LED matrix shall display the red quarantine status, overriding any pattern set by actuator commands. When the last quarantine is released, the matrix shall return to green.
- **FR-2.7:** Each software component shall run as an independent process or container, so that one component failing (for example, the dashboard) does not stop the others on the same node.
- **FR-2.8:** If the HMI Pi becomes unreachable, the Sense HAT Pi shall keep all existing quarantine rules in place until it receives an explicit release order.

3. Real-World Physical or System-Level Inputs and Network State Changes

- *Mandatory Baseline:* The system must use real-world physical or system-level inputs (Sense HAT environmental sensors, IMU, LED matrix, joystick input, etc.) and transmit them across the network to trigger observable state changes or analytics.
- **Write your Project's Specific Functional Requirement(s)** (Specify which physical Sense HAT sensors/actuators or Linux system telemetry metrics your project will sample, at what frequency/threshold, and what observable state change or analytic action is triggered across the network):

Physical / System Inputs Sampled & Rate: _____ Sense HAT temperature, humidity, and pressure sensors sampled at 1 Hz; IMU accelerometer sampled at 10 Hz locally for tamper detection; joystick presses captured as events; inbound connection attempts to

the command service and canary port captured as system-level security inputs.

_____

Network Transmission & Triggered State Change / Analytic Output: _____ Environmental readings are streamed over UDP Socket 1 and update the HMI dashboard live, with threshold violations raising an environmental alarm. Tamper detections and joystick presses are sent over UDP Socket 5, raising or acknowledging alerts on the HMI. Unauthorized connection attempts are reported over TCP Socket 3 and trigger an automated quarantine order on Socket 4, which changes the edge node's firewall state and turns the LED matrix red. _____

List your specific functional requirements here:

- **FR-3.1:** The sensor daemon shall sample temperature (°C), relative humidity (%), and barometric pressure (hPa) once per second and transmit each reading to the HMI Pi over Socket 1.
- **FR-3.2:** The HMI Pi shall raise an environmental alarm when a reading crosses a configurable threshold (default: temperature above 40 °C or humidity above 80%), highlight the affected value on the dashboard, and log the alarm to the NAS.
- **FR-3.3:** The sensor daemon shall sample the IMU accelerometer at 10 Hz and report a tamper event over Socket 5 when the total acceleration magnitude exceeds 1.5 g, indicating the edge node was physically moved or struck.
- **FR-3.4:** A tamper event shall set the LED matrix to amber, display a tamper alert on the HMI dashboard, and be logged to the NAS. Tamper events shall not trigger a quarantine, since they don't identify a network host.
- **FR-3.5:** A joystick center press on the Sense HAT shall be transmitted over Socket 5 as a local operator acknowledgment, clearing active amber alerts on both the LED matrix and the HMI dashboard. Acknowledgment shall not clear an active quarantine.
- **FR-3.6:** Each connection to the canary port and each rejected command on the command service shall be captured with its source IP, source port, and timestamp and reported to the HMI Pi over Socket 3.
- **FR-3.7:** The quarantine controller shall quarantine a host immediately upon a single canary hit, or after three unauthorized command attempts from the same source IP within 60 seconds.
- **FR-3.8:** A successful quarantine shall produce three observable changes within 2 seconds of the triggering event: a new nftables rule on the Sense HAT Pi, a red LED matrix, and a new entry in the HMI dashboard's quarantine list.

4. HMI Pi Touchscreen Live User Interface

- *Mandatory Baseline:* The HMI Pi touchscreen must display a live, updating user interface (GUI, dashboard, or structured terminal UI) that visualizes real-time network/sensor state.
- **Write your Project's Specific Functional Requirement(s)** (Specify the UI framework/technology, Ex: Python Tkinter/PyQt GUI, curses/Rich structured terminal UI, or non-WebSocket local dashboard, the refresh rate, and the exact real-time sensor and network socket metrics displayed on the touchscreen):

UI Framework / Type: _____ Python Tkinter full-screen GUI running natively on the HMI Pi touchscreen, with touch input handled as standard button presses. No web server or WebSockets. _____

Real-Time Metrics & States Visualized (and Refresh Rate): _____ Live temperature, humidity, and pressure; edge node online/offline status and last heartbeat time; per-socket status and message counts; a security event feed; the active quarantine list; and alarm states for environmental, tamper, and security alerts. Sensor values and socket metrics refresh every 1 second, while security events and quarantine changes appear within 500 ms of arrival. _____

List your specific functional requirements here:

- **FR-4.1:** The dashboard shall be built with Python Tkinter and run full-screen on the HMI Pi touchscreen, launching automatically at startup.
- **FR-4.2:** The dashboard shall display the latest temperature (°C), humidity (%), and pressure (hPa) from Socket 1, refreshing once per second. Any value over its alarm threshold shall be highlighted in red.
- **FR-4.3:** The dashboard shall show the Sense HAT Pi's status as online or offline based on Socket 5 heartbeats, along with the time since the last heartbeat. The status shall change to offline within 6 seconds of heartbeats stopping.
- **FR-4.4:** The dashboard shall display a socket status panel listing each of the project's seven sockets with its transport, port, connection state (listening, connected, or down), and a running message count, refreshing once per second.
- **FR-4.5:** The dashboard shall display a scrolling security event feed showing each event's timestamp, type, and source IP. New events shall appear within 500 ms of reaching the controller.
- **FR-4.6:** The dashboard shall display the active quarantine list, showing each quarantined IP, the reason, and the time it was quarantined, with a touch-sized Release button on each entry that sends a release order through the controller.
- **FR-4.7:** The dashboard shall provide touch buttons for sending authorized actuator commands to the Sense HAT Pi (at minimum, setting the LED matrix to a test pattern and clearing it) and shall display each command's acknowledgment result.
- **FR-4.8:** The dashboard shall display a system status banner showing the current overall state (normal, alert, or quarantine active), matching the color of the Sense HAT LED matrix.
- **FR-4.9:** All touch controls shall be at least 48 × 48 pixels so they can be operated reliably by finger on the touchscreen.

5. Persistent Structured Data Logging on Remote Storage

- *Mandatory Baseline:* The system must log persistent, structured data using a remote storage device (writing records to the database on the NAS, publishing/subscribing via an MQTT broker, or automating encrypted remote file backups using SFTP/SCP.)

- **Write your Project's Specific Functional Requirement(s)** (Identify which remote storage mechanism(s) your project uses, the exact schema/format of the structured data, Ex: SQL table columns, JSON payload structure, or CSV log fields, and when records are written):

Remote Storage Mechanism Selected (NAS DB / MQTT / SFTP / SCP): __ NAS database (SQL), accessed by the HMI Pi logger over TCP Socket 7.

_____

Structured Data Schema / Table / Topic & Logging Trigger: _____ Three tables. telemetry stores every sensor reading, written in batches every 10 seconds. security_events stores each detection and alert, written as soon as the event is received. quarantine_actions stores each quarantine and release order and its outcome, written as soon as the enforcement agent confirms or rejects it. _____

List your specific functional requirements here:

- **FR-5.1:** The HMI Pi logger shall store all persistent data in the classroom NAS database over TCP Socket 7, using the three tables defined above.
- **FR-5.2:** Every telemetry reading received on Socket 1 shall be written to the telemetry table. Readings shall be inserted in batches every 10 seconds to limit database round trips.
- **FR-5.3:** Every canary hit, unauthorized command, tamper detection, environmental alarm, and operator acknowledgment shall be written to security_events within 2 seconds of being received by the HMI Pi.
- **FR-5.4:** Every quarantine and release order shall be written to quarantine_actions with its result once the enforcement agent responds, so that each quarantine can be traced back to the event that caused it.
- **FR-5.5:** Both the edge node's timestamp and the HMI Pi's receive timestamp shall be stored for telemetry, allowing network delivery delay to be measured from the logs.
- **FR-5.6:** If the NAS is unreachable, the logger shall buffer records in memory (up to 10,000 records) and write them in order once the connection is restored, without stopping the rest of the system.
- **FR-5.7:** The team shall be able to retrieve the complete quarantine history for a demo session with a single SQL query joining quarantine_actions and security_events.

6. Docker Containerization & Deployment

- *Mandatory Baseline:* At least one core component or microservice of your project architecture must be packaged and deployed using Docker (using Dockerfile, Docker Compose, or Docker Swarm across the nodes).
- **Write your Project's Specific Functional Requirement(s)** (Identify which specific microservice(s) will be containerized, which Pi node(s) will host the container, the orchestration method (Dockerfile, Docker Compose, or Docker Swarm) and how container ports are mapped to the host network):

Containerized Component and Orchestration Tool: _____ The quarantine controller, packaged with a Dockerfile (based on python:3.11-slim for ARM64) and deployed with Docker Compose. _____

Host Node and Port Mapping: _____ Runs on the HMI Pi using Docker's default bridge network. Container port 5020/tcp is published to host port 5020 on all interfaces for incoming security events (Socket 3). Container port 5070/tcp is published to 127.0.0.1:5070 only, for the local dashboard control channel. Outbound connections to the enforcement agent (Sense HAT Pi port 5030) go out through Docker's NAT and appear to come from the HMI Pi's IP. _____

List your specific functional requirements here:

- **FR-6.1:** The quarantine controller shall be packaged as a Docker image built from a Dockerfile in the team repository and run as a container on the HMI Pi.
- **FR-6.2:** The controller shall be deployed with a docker-compose.yml file, so that docker compose up -d builds and starts it with no other manual steps.
- **FR-6.3:** The container shall publish port 5020/tcp to the host so the Sense HAT Pi event reporter can reach it at the HMI Pi's IP address.
- **FR-6.4:** The container shall publish port 5070/tcp to 127.0.0.1 only, so the dashboard can receive quarantine updates and send operator release requests while the port stays unreachable from the network.
- **FR-6.5:** Runtime settings, including the Sense HAT Pi IP, enforcement agent port, detection thresholds, and allowlist, shall be supplied as environment variables in docker-compose.yml rather than hardcoded in the image.
- **FR-6.6:** The container shall use the restart: unless-stopped policy so it restarts automatically after a crash or a reboot of the HMI Pi.
- **FR-6.7:** Controller logs shall be written to standard output so they can be viewed live with docker compose logs -f during the demo.

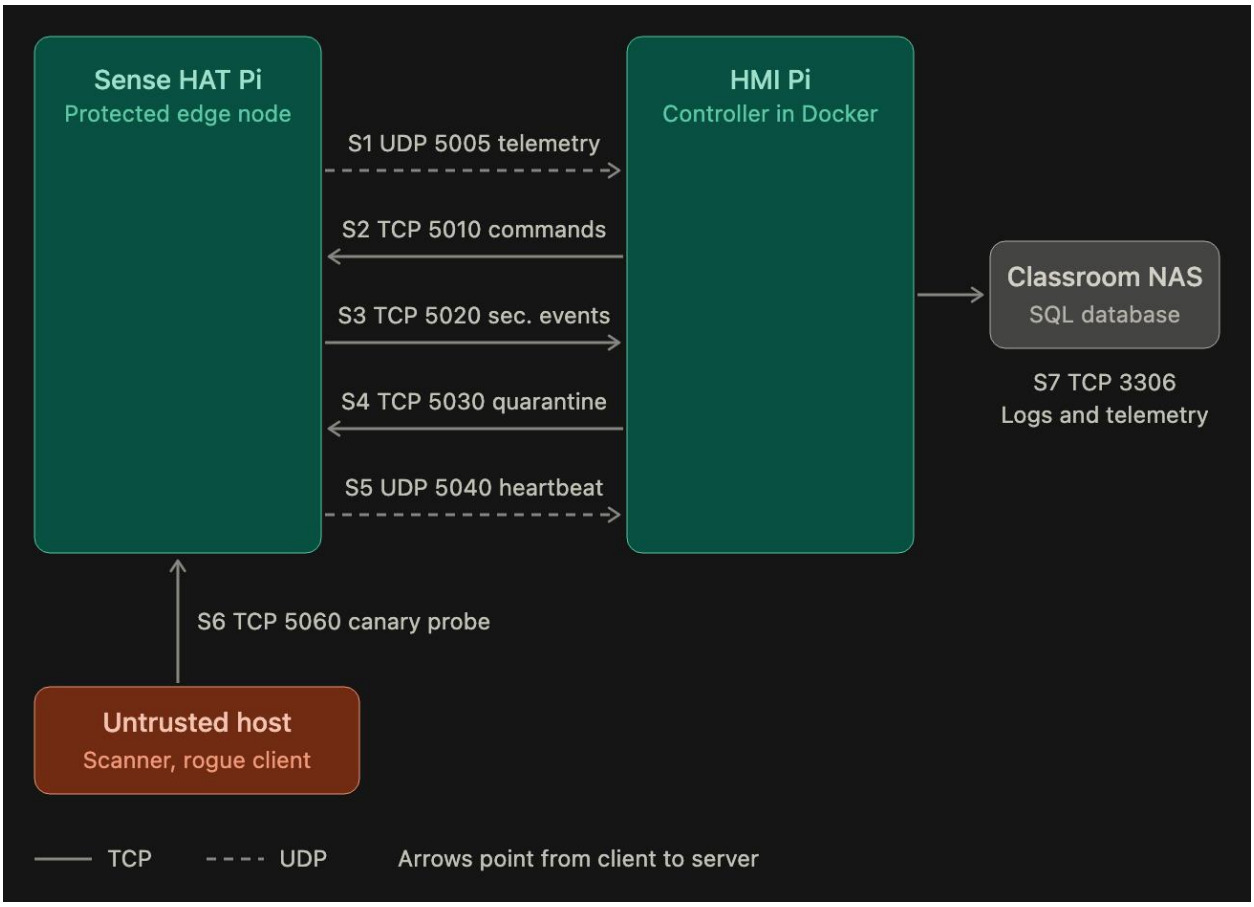
Question 2: Review your six sockets above. Explain why your team chose **TCP** versus **UDP** for each socket based on connection overhead, reliability, message boundaries, and latency requirements.

Answer:

We chose UDP for the telemetry stream (Socket 1) and the heartbeat and joystick channel (Socket 5) because both carry frequent, time-sensitive messages where the newest value matters more than any single old one. If a telemetry datagram is lost, TCP would retransmit it and hold back every reading behind it until it arrived, while UDP lets the HMI

simply display the next reading a second later, and our sequence numbers let us detect any gaps. UDP also avoids per-message acknowledgment and retransmission overhead and preserves message boundaries, so each datagram is one complete JSON reading. A lost heartbeat is harmless because the HMI only marks the edge node offline after three are missed, and a lost joystick press is an acceptable tradeoff since the operator can simply press again. We chose TCP for the actuator command channel (Socket 2), security event reporting (Socket 3), quarantine enforcement (Socket 4), remote logging (Socket 7), and the dashboard control channel (Socket 8) because every message on these sockets must arrive, in order, and often needs a confirmation: a dropped security event or quarantine order would leave an attacker with access, and a lost log record would break the audit trail. These connections are persistent, so the three-way handshake happens once rather than per message, and because TCP is a byte stream without message boundaries, we frame each message as newline-delimited JSON. TCP's reliability does not make it secure, since it provides no encryption or authentication, which is why TLS is listed as out of scope. Finally, the canary listener (Socket 6) uses TCP because scanners typically probe TCP ports, and the completed handshake confirms the connecting source IP is genuine, whereas UDP source addresses can be easily spoofed and could trick the system into quarantining an innocent host.

PASTE SCREENSHOT OF PART 2 BELOW:
(Screenshot of your project document or architecture diagram showing your 6 project specific Functional Requirements)



Part 3: Specific Non-Functional Requirements Specification (Section 3)
Whereas functional requirements specify *what* the system does, **Non-Functional Requirements** specify *how* the system operates, such as: governing system reliability, security, network isolation, auditability, deployment speed, and performance characteristics.

In this section, your team will translate the mandatory constraints in Section 3 of the *Project Description* into concrete operational rules and verification commands for your specific project.

1. GitHub Version Control

- *Mandatory Baseline:* All source code, configuration files, and documentation must be maintained in a shared GitHub repository with regular commits from both team members. Raspberry Pi micro-SD cards can fail unexpectedly, so no sole copy of project code may reside exclusively on the sandbox hardware.
- **Your Team's Specific Version Control Plan** (Define your team's branching/commit workflow, how both partners will contribute commits, and how configuration files/scripts are synced off the Pis at the end of every work session):

GitHub Repository Public Link (URL): ___ https://github.com/isaacgentry28/Factory-Network-Monitor

Repository Branching & Commit Policy: The team plans to make commits after major changes or additions that affect the functionality of the system.
Backup / Off-Hardware Sync Procedure: scripts and files will be downloaded into a USB. Thumb drive for backup storage which is regularly updated. Also, for updates across the Pis, git commands to put the executable scripts and appropriate files onto the Pi will be used, as well as possible SFTP commands.
Write your specific Version Control requirements here:

2. Unprivileged Socket Port Allocation (1025–65535)

- *Mandatory Baseline:* All project sockets must bind to unprivileged socket ports (1025–65535) unless explicitly justified.
- **Your Team's Specific Port Mapping** (List the exact port numbers between 1025 and 65535 assigned to your custom services, and provide an explicit justification if your project interacts with any standard privileged port <= 1024 such as SSH/SFTP port 22):
- **Assigned Custom Unprivileged Ports (1025–65535):** UDP 5005 (telemetry)

TCP 5010
TCP 5020
TCP 5030
UDP 5040
TCP 5060
TCP 5070

Privileged Port Justification (if any): no ports at or below 1024
Write your specific Port Mapping requirements here:
NFR 2.1: Every project server socket shall bind to one of the ports listed above, all within 1025-65535.
NFR 2.2: All client sockets shall use operating-system-assigned temporary ports and shall not bind to fixed ports.
NFR 2.3: Port number shall be defined in one shared configuration file (or environment variables) rather than hardcoded in individual scripts.
NFR 2.4: The enforcement agent shall run with root privileges only because it modifies nftables. It still binds an unpriveleged port (5030), and no other project component shall run as root.

3. Managing Network Interfaces (eth0) and (wlan0)

- *Mandatory Baseline:* The primary wired management interface (eth0) IP configuration and default routes on both Raspberry Pis must not be altered or bridged in a way that severs SSH access or disrupts neighboring sandbox tables. (wlan0) on the Raspberry Pis can be used, if needed.
- **Your Team's Specific Network Interface Plan** (Specify how your services bind to eth0 and/or wlan0 without modifying default routes or severing SSH management access):

eth0 Interface Binding Plan:
All project traffic uses eth0 on both Pis. Services bind to the Pi's eth0 IP address, read from configuration.

wlan0 Interface Binding Plan: no services bind to it I believe
Write your specific Network Interface requirements here: here:
NFR-3.1: The project shall not modify the IP configuration, DHCP settings, or default route of eth0 on either Pi.
NFR-3.2: The enforcement agent shall write rules only inside its own nftables table (inet cps_quarantine). It shall add or remove only per-IP drop rules and shall never flush or replace the system's full ruleset.
NFR-3.3: The enforcement agent shall refuse any quarantine order targeting the HMI Pi, the team's management host, or the classroom gateway, so SSH access to the Sense HAT Pi is never cut off.
NFR-3.4: Docker on the HMI Pi shall use its default bridge network only, which leaves eth0's address and the default route unchanged.
NFR-3.5: If a firewall change locks out SSH, all quarantine rules shall be removable with one command run from the Pi's local console: sudo nft delete table inet cps_quarantine.

4. Classroom LAN Confinement

- *Mandatory Baseline:* Any active network probing or discovery must remain strictly confined to the classroom LAN.
- **Your Team's Specific Subnet Boundary** (Record the exact classroom CIDR subnet(s) your application traffic, broadcasts, or discovery probes are restricted to):

Allowed Target IP / Subnet Range(s): class room subnet(192.168.5.0/24)
Write your specific Subnet Bounary requirements here: here:
NFR-4.1: All project traffic shall stay within the classroom subnet. The only destinations shall be the HMI Pi, the Sense HAT Pi, and the classroom NAS.
NFR-4.2: The system shall detect threats passively, through the canary listener and command-service rejections. It shall never actively scan or probe other hosts.
NFR-4.3: Any attack testing, such as nmap scans or unauthorized commands, shall target only the team's own Sense HAT Pi, run from a team-owned host, for example nmap -p 5060 <SenseHAT_IP>.
NFR-4.4: The quarantine controller shall reject any quarantine order whose target IP is outside the classroom subnet.
NFR-4.5: All UDP traffic shall be unicast. No broadcast or multicast shall be used.

5. Live Linux CLI Auditability and Verification

- *Mandatory Baseline:* During the final demo, your team must be able to prove and audit your running architecture using standard networking Linux CLI tools. This includes verifying all 6+ active sockets and PIDs by ss and lsof, demonstrating link/route health using ip/ping/ethtool, and measuring or validating network traffic behavior.
- **Your Team's Specific CLI Verification Commands** (Write the exact Linux CLI commands your team will execute live during the Week 15 Expo to prove your sockets, processes, and links are operating as designed):

Command to verify all 6+ project sockets & PIDs via ss: sudo ss -tulpn | grep -E ':(5005|5010|5020|5030|5040|5060|5070)\b'
sudo ss -tnp state established
Command to verify project port ownership via lsof: sudo lsof -nP -i :5005,5010,5020,5030,5040,5060,5070,3306
Commands to verify interface, route, and physical link health (ip, ping, ethtool):
ip -br addr show eth0
ip route show default
ip -s link show eth0
ping -c 4 <SenseHAT_IP> (from the HMI Pi)
ping -c 4 <NAS_IP>
sudo ethtool eth0 (check Speed, Duplex, and "Link detected: yes")
Write your specific CLI Verification requirements here: here:
NFR-5.1: During the demo, ss -tulpn and lsof shall show every project socket with its owning process and PID.
NFR-5.2: On the HMI Pi, ports 5020 and 5070 shall show as owned by docker-proxy on the host. docker compose ps shall be used to confirm the controller container behind them.
NFR-5.3: Traffic behavior shall be shown live with tcpdump, for example sudo tcpdump -i eth0 -n udp port 5005 -c 10 for telemetry, with readings arriving at 1 per second.
NFR-5.4: Quarantine enforcement shall be shown live with sudo nft list table inet cps_quarantine before and after a canary hit.
NFR-5.5: All demo verification commands shall be saved in docs/demo_commands.md in the repository.

6. Under 3-Minute Reboot/Launch Reproducibility

- *Mandatory Baseline:* The project repository must include a comprehensive README.md with an architecture/socket diagram and automated startup scripts (or docker compose up commands) allowing the system to be launched cleanly from a fresh reboot in under 3 minutes.
- **Your Team's Specific Startup Automation Plan** (Describe the startup script, systemd service, or docker compose up workflow your team will build so the entire distributed system launches cleanly within 180 seconds of a cold reboot on both Pis):
- **Automated Startup Mechanism (HMI Pi & Sense HAT Pi):**

Sense HAT Pi: Each daemon runs as a systemd service, grouped under one cps-edge.target. The services are enabled to start at boot after network-online.target, with Restart=always.
HMI Pi: The Docker service is enabled at boot, and the controller uses restart: unless-stopped. The receiver and logger run as systemd services. The Tkinter dashboard launches full-screen through a desktop autostart entry.
Fallback: A manual scripts/start_all.sh script is also provided.

Target Boot-to-Operational Time: 90 seconds (< 180 seconds)
Write your specific Startup Automation requirements here:

NFR-6.1: After a cold reboot of both Pis, the full system shall be operational within 180 seconds, with a target of 90 seconds. “Operational” means the dashboard shows live telemetry and the edge node shows as online.
NFR-6.2: No manual commands shall be needed after boot.
NFR-6.3: Components shall start in any order. Every client shall retry its connection every 2 seconds until its server is available.
NFR-6.4: The repository shall include a one-time scripts/install.sh for each node that installs dependencies and enables the systemd services.
NFR-6.5: The README shall include an architecture and socket diagram, the socket allocation table, and step-by-step launch instructions.
NFR-6.6: The team shall time a full cold reboot before the Expo and record the result in the README.

PASTE SCREENSHOT OF PART 3 BELOW:
(Screenshot of your completed Section 3 Non-Functional Requirements specification in your project document or README.md, plus a terminal screenshot running ip -br addr and ss -tulpn on your sandbox Pis)

Part 4: Initial Socket Allocation Table Draft (Section 4 Preview)
To verify that your specific requirements in **Parts 1, 2, and 3** form a cohesive network architecture, complete your initial draft of the **Required Socket Allocation Table** from **Section 4** of the *Project Description* (remembering to use unprivileged ports 1025–65535 and at least 6 distinct TCP/UDP sockets):

Socket #	Transport (TCP/UDP)	Server Node / IP / Port	Client Node / IP	Service / Component Name	Purpose & Payload Description
Socket 1 (Example)	TCP	HMI Pi 192.168.5.X2 : 8001	Sense HAT Pi 192.168.5.X3	IMU DataStream	High-rate 10Hz temperature/IMU JSON datagrams
Socket 1	UDP	HMI Pi	Sense. HAT Pi	Telemetry Stream	1 Hz JSON datagrams: node ID, timestamp, temperature, humidity, pressure, sequence number
Socket 2	TCP	Sense HAT Pi	HMI Pi	Actuator Command Service	
Socket 3	TCP	HMI Pi	Sense HAT	Security Event Reporting	JSON events: canary_hit or unauthorized_command, source IP and port, timestamp
Socket 4	TCP	SenseHAT Pi	HMI Pi	Quarantine Enforcement Agent	JSON orders: action (quarantine or release), target IP, reason, order ID; JSON confirmation returned once the nftables rule is applied
Socket 5	UDP	HMI Pi 192.168.5.12 : 5040	Sense HAT Pi 192.168.5.13	Heartbeat and Joystick Events	JSON heartbeat every 2 s: node ID, timestamp, uptime, active quarantine count; JSON datagram on each joystick press: event type, direction, timestamp
Socket 6	TCP	Sense HAT Pi 192.168.5.13 : 5060	Any untrusted host	Canary Listener	No payload expected; connection source IP and port are recorded, the connection is closed, and a canary_hit event is sent over Socket 3

Question 4: Trace the end-to-end lifecycle of a single piece of sensor or system data in your project, starting from physical acquisition on the Sense HAT Pi, across the Ubiquiti managed table switch, into the HMI Pi user interface, and finally into persistent remote storage. Which exact sockets from your table above are traversed at each step?
Answer:

On the Sense HAT Pi, `sensor_daemon.py` reads the temperature, humidity, and pressure sensors over the Pi's internal I2C bus once per second and packages the values

into a JSON object with the node ID, a timestamp, and a sequence number. The daemon sends this object as a single UDP datagram over **Socket 1** to the HMI Pi at port 5005. The datagram leaves the Sense HAT Pi's eth0 interface and passes through the Ubiquiti managed table switch, which forwards the Ethernet frame to the HMI Pi's port based on its destination MAC address; since both Pis are on the same subnet, no routing is involved. On the HMI Pi, `telemetry_receiver.py` receives the datagram, parses the JSON, checks the sequence number for gaps, and places the reading on a queue that the Tkinter dashboard reads on its 1-second refresh timer, updating the temperature display and highlighting it if it exceeds the alarm threshold. The reading is also added to `nas_logger.py`'s batch, and every 10 seconds the logger writes the batch over **Socket 7**, a persistent TCP connection that again crosses the Ubiquiti switch, to the classroom NAS database at port 3306. The NAS stores the reading as a row in the `telemetry` table with both the Sense HAT Pi's original timestamp and the HMI Pi's receive timestamp, which allows network delivery delay to be measured from the logs. In total, the reading traverses Socket 1 (UDP) from the edge node to the HMI and Socket 7 (TCP) from the HMI to remote storage, crossing the managed switch once on each hop.